

Multi-Agent Reinforcement Learning Algorithms for Safe Hovering and Navigation on Crazyflie Drones

**Machine Intelligence & Robot Learning (MIRL) Lab
University of Ottawa**

**CSI4900
Fall 2025**

Supervisor: Colin Bellinger

Michael O'Sullivan - 300228801
Aydin Yalcinkaya - 300238613

Abstract

We present a multi-agent reinforcement-learning (MARL) approach for learning safe take-off and hovering policies for Crazyflie drones¹. Starting with a single-drone simulation environment, we trained a drone to navigate to and hover at a target in XYZ using proximal policy optimization (PPO)². We then extended the simulation to use multi-agent PPO (MAPPO)³ on two drones to learn safe XYZ navigation and hover without collisions. This resulted in successful simulation results for a variety of test targets, with the drones navigating to the target and hovering in close proximity without crashes or collisions. To support the transition from simulation to hardware (sim2real), we implement domain randomization to simulate the noisy sensor output and physical variations associated with Crazyflie drones. After training a PPO policy for one drone and a MAPPO policy for two drones, we evaluated our policies in simulation and tested each policy on real Crazyflie drones. Our PPO policy performed with moderate success when applied to a real Crazyflie drone, however, while our MAPPO policy was experimentally successful in simulation, it failed to perform on real Crazyflie drones.

1.0 Introduction

The goal of this project was to develop a reinforcement learning environment and training schedule that would result in successful collaborative behaviour between multiple drones. For a relatively simple task, we chose the goal of learning two drones to navigate to and hover at a shared target in 3-dimensional space. Our initial simulation was done using the MuJoCo⁴ physics simulator with a Gymnasium⁵ reinforcement learning environment wrapper. While this approach worked for initial testing (e.g. single drone thrust control on the Z-axis), we quickly learned that training becomes time-consuming for large learning tasks since Gymnasium environments are

CPU-bound. We therefore switched to a Jax⁶-based training environment and used MuJoCo playground⁷ for physics, both of which support fast, vectorized computations on the GPU. A single-drone PPO policy and two-drone MAPPO policy were trained using this method and evaluated in simulation prior to testing on real drones. We additionally performed experimental testing on the real Crazyflie drones to find appropriate control limits and control scaling for the real drones. The single-drone PPO policy showed moderate success navigating to and hovering at close targets. The two-drone MAPPO policy, however, had inconsistent and erratic behaviour despite its success in our simulated evaluation and would often trigger safety control overrides early in the rollout.

2.0 Problem Formulation

Our MAPPO approach uses deep reinforcement learning, therefore we train two deep neural networks: the actor (policy) and the critic (state estimator for policy advantages). The neural networks are multilayer perceptrons with two hidden layers. Each hidden layer in the actor has 128 neurons and each hidden layer in the critic has 256 neurons due to the larger input size of the global observation. The actor (drone controller) receives a per-drone observation and outputs continuous action distributions for drone thrust, roll, pitch, and yaw $a_t \in R^4$ at time t . The critic receives a global observation for all drones and outputs a scalar estimate for the value of the current state. Our per-drone observation $o_t \in R^{34}$ at time t consists of the drone’s position in world space R^3 , rotation matrix R^9 , linear velocity in world and drone body space R^6 , angular velocity R^3 , relative position to target in drone body space R^3 , relative position to all drones in body space R^6 , and previous action R^4 . We use a rotation matrix rather than a quaternion to represent rotation in order to avoid ambiguity⁸. We have experimented with two drones, so for relative

drone positions each drone sees 6 features / 2 positions (currently drones see their own relative position $[0, 0, 0]$, we aim to eliminate this in the future). Our reward function at time t is defined as:

$$r_t = r_t^{survival} + r_t^{targetProximity} + r_t^{distImprovement} + r_t^{droneProximity} + r_t^{failure}$$

$r_t^{survival}$ = step survival constant (+0.001)

$r_t^{targetProximity}$ = function $(1 - \tanh(dist))$ of proximity to the target (weight +0.01)

$r_t^{distImprovement}$ = function $prevDist - currentDist$ of target distance improvement (weight +/-1.0)

$r_t^{droneProximity}$ = pairwise proximity function $1 - dist/threshold$ if $dist < threshold$ of inter-drone distances (weight -0.01)

$r_t^{failure}$ = constant penalty for any of the drone failure conditions: crashing, colliding, or leaving the training boundary (-1.0)

Episodes always run until the maximum episode length unless interrupted by any of the above failure conditions (crash/collision/boundary exit), in which case the $r_t^{failure}$ penalty is applied.

Advantages estimates are computed using generalized advantage estimation (GAE)⁹, where if an episode runs until the maximum episode length, it is truncated and bootstrapped with a predicted next-state value instead of a terminal return. In this setup where terminal episodes indicate failure, our fundamental success criteria was to have 2+ drones successfully navigate to and hover at a target in XYZ without colliding, crashing, or leaving the bounds throughout the episode.

3.0 System Design

MuJoCo provides many robot XML models and scenes for simulation, among them is the Crazyflie drone scene which comes equipped with physics for drone mass, actuators, inertia, and more. We have customized the scene to contain two drones with different colours for clarity and added/adjusted other elements as well, such as an XYZ axis indicator. We also removed collision meshes since we end episodes if a collision (which is detected via proximity, not physical collision) occurs and because the collision computations were making training much slower. This XML scene is used for the physics steps in our environment: when an agent takes an action, the action is applied to the corresponding drone's thrust and torque actuators, which causes the drone's physical state (position, rotation, velocity...) to change. In order to make our policy robust for sim2real (differences between simulation and real hardware) we added domain randomization to our training environment, perturbing drone dynamics such as mass and actuators, as well as observation values such as position and velocity. Real Crazyflie drones use a 3 axis accelerometer/gyroscope as well as a high precision pressure sensor to determine elevation. In order to determine its position in XY space, we attach a FlowDeck (optical flow sensor). The drones output their sensor observations to logs, so we simply register and track the logs we are interested in. The control loop, which we set at 50Hz to mimic the MuJoCo simulations, polls the latest data from the logs and supplies it to our (MA)PPO model. The output of the model is an action, which we perform some post-processing on (control scaling and clamping). This action is then sent over radio to each of the drones. In an effort to reduce damage to the lab and hardware, we implemented several safety precautions, such as a controlled descent if one of the drones exits a bounded range or is coming to the end of the episode time limit. Manual override is also present, which completely stops the drone's propellers.

4.0 Methods

We experimented with varying hyperparameters for our MAPPO use-case, relevant MAPPO hyperparameters are outlined below.

1. Learning rate: Linearly annealed from $3e-3$ (start) to $0.25 * 3e-3$ (end) during training.
2. GAE discounting: discount factor of 0.995 and a lambda of 0.95.
3. PPO clip epsilon: 0.2.
4. Entropy coefficient: $1e-3$.
5. Coefficient for the value function loss: 0.5
6. Gradient clipping threshold: 0.5.
7. Minibatching: 2 epochs and 16 minibatches.
8. Maximum episode length: 1500 steps.

Our training consists of 2048 environments running in parallel, with the policy sampling from each one at every step. We take 128 steps across each environment before collecting a rollout and updating the policy. The policy is updated a total of ~ 1000 times during training, which takes about 50 minutes when running in a Google Colab¹⁰ T4 GPU runtime. To ensure policy robustness across a diverse set of states and targets, each environment receives a random seed / RNG when reset, which is used to randomize the starting states (position, rotation, velocity, angular velocity) of the drones and to randomize the position of the target, within certain thresholds. Position randomization is uniform on $[-2, 2]$ metres for X/Y positions, and $[0.1, 2]$ metres for Z position. The rest of the drone's state values are normally distributed with a mean of 0 and with standard deviations as follows:

1. Rotation angle: $\sigma = \text{PI} / 8$ (applied to a random axis)
2. Linear Velocity: $\sigma = 0.1$
3. Angular Velocity: $\sigma = 0.05$

To make our policy robust against physical differences and observation noise on real hardware (sim2real) we also implement domain randomization, which is also determined by the environment seed. We add noise to the drone's mass, inertia, and actuators at each reset and to the drone's observations at each step. This noise is also normally distributed - for the drone physics (e.g. mass) the distribution is centered at the original value with a standard deviation of 0.1 times the original value. For observations, noise distributions are centered at the original observation with noise standard deviations as follows:

1. Position: 0.01
2. Velocity: 0.005
3. Angular velocity: 0.001
4. Quaternion angle: 0.001

Domain randomization noise is clamped beyond three standard deviations, this range is arbitrary but was set in an attempt to keep randomization within a reasonable range. Training figures for our PPO and MAPPO model are presented below (Figure 1 and 2).

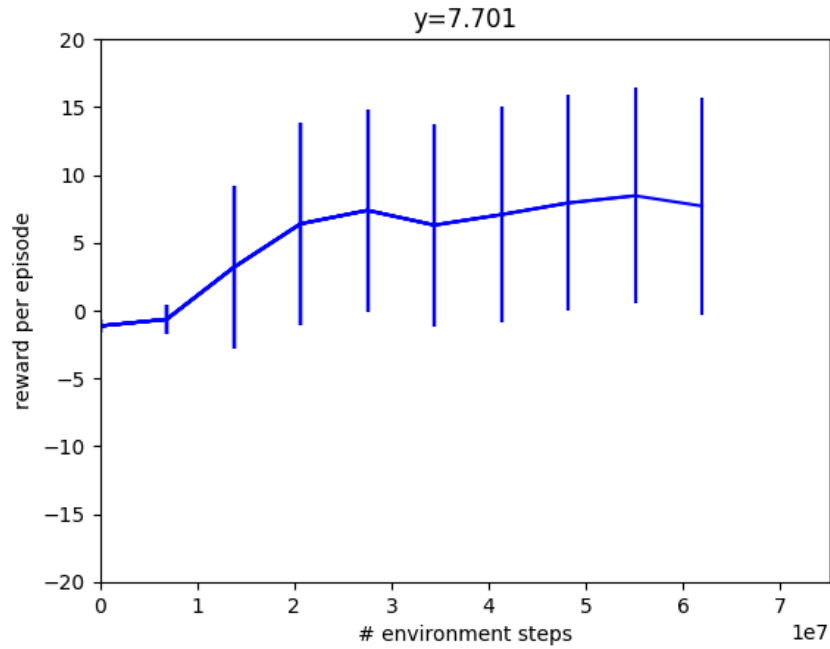


Figure 1: PPO training for a single drone, 2048 parallel environments, 60 million environment steps total, ~20 minutes training time

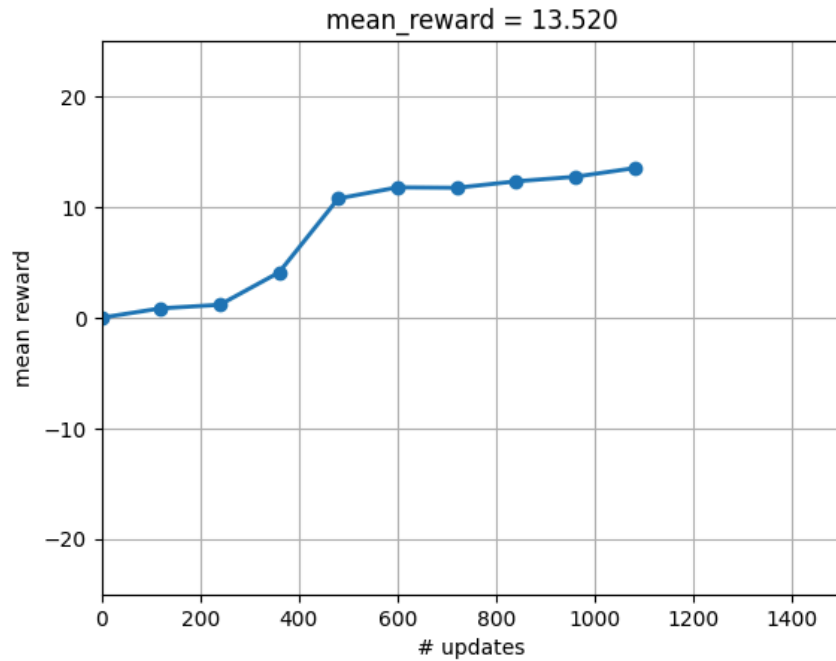


Figure 2: MAPPO training for two drones, 2048 parallel environments, ~300 million environment steps total (128 steps per update), ~50 minutes training time

5.0 Experiments and Results

After training, we evaluated our policy in simulation (with domain randomization) and on real drones. In simulation, we test a variety of targets with varying seeds, testing robustness to a variety of goal and noise scenarios. Our model consistently failed for target positions that were on the boundary or outside of the training ranges (e.g. very close to the ground or very far away). For these targets, drones would end up crashing or colliding at varying stages of the episode. Our model did, however, consistently succeed for targets in the mid ranges (within $\sim 2\text{m}$). These mid-range targets had varying degrees of success, with the model generally performing better for closer targets. Full results are shown in Table 1, in which we see the model consistently failing for target positions (in metres) $[0, 0, 0.2]$ and $[7, 7, 7]$ and succeeding for the mid-range targets. This indicates that while our policy struggles to generalize for extreme targets, it seems to perform consistently within each target test despite the domain randomization, seen by the consistently low return standard deviation across each test.

Target #	Target (x,y,z)	Mean Return	Std Return	Mean Steps	Failures
0	[0.0, 0.0, 0.2]	5.95	0.09	703.9	64/64
1	[0.3, 0.0, 1.0]	55.13	0.04	3000	0/64
2	[-0.3, 0.0, 1.0]	49.63	0.1	3000	0/64
3	[0.0, 0.3, 1.0]	53.33	0.11	3000	0/64
4	[0.0, -0.3, 1.0]	53.62	0.09	3000	0/64
5	[0.5, 0.5, 1.2]	57.37	0.02	3000	0/64
6	[-0.5, 0.5, 1.2]	53.61	0.08	3000	0/64
7	[0.5, -0.5, 1.2]	56.82	0.6	3000	0/64
8	[-0.5, -0.5, 1.2]	52.4	0.09	3000	0/64
9	[0.0, 0.0, 2.0]	36.97	0.01	3000	0/64
10	[1.0, 0.0, 2.0]	55.53	0.07	3000	0/64
11	[-1.0, 0.0, 2.0]	21.06	0.01	3000	0/64
12	[0.0, 1.0, 2.0]	35.94	0.02	3000	0/64
13	[0.0, -1.0, 2.0]	35.19	0.01	3000	0/64
14	[2.0, 2.0, 2.0]	50.34	0.02	3000	0/64
15	[-2.0, 2.0, 2.0]	11.74	0	3000	0/64
16	[2.0, -2.0, 2.0]	52.41	0.02	3000	0/64
17	[-2.0, -2.0, 2.0]	13.64	0.01	3000	0/64
18	[0.0, 0.0, 3.0]	14.94	0.01	3000	0/64
19	[-3.0, 0.0, 3.0]	7.68	0	3000	0/64
20	[0.0, -3.0, 3.0]	12.82	0.6	2982.9	1/64
21	[3.0, 3.0, 3.0]	42.05	0.02	3000	0/64
22	[5.0, 5.0, 5.0]	19.07	0	3000	0/64
23	[7.0, 7.0, 7.0]	9.21	0.28	2555.4	64/64

Table 1: Evaluation of MAPPO policy in simulation for varying targets and seeds

We additionally applied our policies to real drones with varying degrees of success. Our single-drone PPO model was able to successfully navigate to and hover at close targets. Compared to a manual hover using a PID controller, the PPO-controlled drone would oscillate more about the target (drift and overcorrect). We later applied our MAPPO model to two drones with less success. While the drones would initially steer towards the target, one or both of the drones would often leave the safety boundary and trigger a control override, ending the episode.

6.0 Discussion

While both agents performed well in simulation, the MAPPO model performed considerably worse on real hardware. One possible reason is the use of a third-party PPO algorithm from Brax¹¹ compared to a custom implementation for MAPPO. The Brax PPO algorithm provides helpful features such as observation standardization, in which new observations are standardized using a historical distribution of previous observations. This was missing in our MAPPO implementation, mostly due to time constraints. It is also possible that a centralized critic is less effective for this task compared to a decentralized per-drone critic, given that the value of drone states are not tightly coupled. For example, a drone might be successfully hovering near the target but would receive a lower state-value estimation from the global critic if the other drone crashes, so perhaps a decentralized critic would be more effective for this task. We also noticed significant differences in control when comparing our simulated drones to real ones. For example, a thrust that hovers the drone in simulation accelerates a real drone into the ceiling. In the documentation for our MuJoCo model, it is mentioned that the control limits are chosen somewhat arbitrarily¹², so taking more time to understand the differences in dynamics between the MuJoCo simulation and real drones would likely yield better sim2real results.

7.0 Conclusions and Future Work

Throughout this project, we have learned the fundamentals of reinforcement learning and have explored a specific multi-agent application on drone robotics. While the trained multi-agent policy was unsuccessful on real drones, we did manage to verify its functionality in simulation, as well as apply our single-agent policy to hardware successfully. To further optimize our MAPPO model, we could experiment with observation standardization and the use of a decentralized critic. For an extension to the existing behaviour, we could add navigational obstacles or adjust drone behaviour to be more collaborative or competitive (e.g. chase or coordinated flight). Adding more drones or making drones follow a path instead of flying to a target could be other, potentially straightforward extensions. Overall, there are many avenues of improvement and future work for this project and we hope to explore them further.

8.0 References

1. *Crazyflie 2.1* | Bitcraze. (2024). [Bitcraze.io](https://www.bitcraze.io).
<https://www.bitcraze.io/products/old-products/crazyflie-2-1/>
2. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017, August 28). *Proximal Policy Optimization Algorithms*. ArXiv.org. <https://arxiv.org/abs/1707.06347>
3. Yu, C., Velu, A., Vinitsky, E., Gao, J., Wang, Y., Bayen, A., & Wu, Y. (2022, November 4). *The Surprising Effectiveness of PPO in Cooperative, Multi-Agent Games*. ArXiv.org. <https://doi.org/10.48550/arXiv.2103.01955>
4. Todorov, E., Erez, T., & Tassa, Y. (2012, October 1). *MuJoCo: A physics engine for model-based control*. IEEE Xplore. <https://doi.org/10.1109/IROS.2012.6386109>

5. Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U., Cola, D., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., Perez-Vicente, R., Pierré, A., Schulhoff, S., Tai, J. J., Tan, H., & Younis, O. G. (2024). *Gymnasium: A Standard Interface for Reinforcement Learning Environments*. ArXiv.org. <https://arxiv.org/abs/2407.17032>
6. Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: composable transformations of Python+NumPy programs*. <http://github.com/jax-ml/jax>
7. Zakka, K., Tabanpour, B., Liao, Q., Haiderbhai, M., Holt, S., Luo, J. Y., Allshire, A., Frey, E., Sreenath, K., Kahrs, L. A., Sferrazza, C., Tassa, Y., & Abbeel, P. *MuJoCo Playground*. (2025). ArXiv.org. <https://arxiv.org/abs/2502.08844>
8. Zhou, Y., Barnes, C., Lu, J., Yang, J., & Li, H. (2018). On the Continuity of Rotation Representations in Neural Networks. *ArXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.1812.07035>
9. Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2018). High-Dimensional Continuous Control Using Generalized Advantage Estimation. *ArXiv:1506.02438 [Cs]*. <https://arxiv.org/abs/1506.02438>
10. Google. (2025). Google Colaboratory. <https://colab.research.google.com/>
11. C. Daniel Freeman, Frey, E., Raichuk, A., Sertan Girgin, Igor Mordatch, & Olivier Bachem. (2021). Brax -- A Differentiable Physics Engine for Large Scale Rigid Body Simulation. *ArXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.2106.13281>
12. Zakka, K., Tassa, Y., & MuJoCo Menagerie Contributors. (2022). *MuJoCo Menagerie: A collection of high-quality simulation models for MuJoCo*. https://github.com/google-deepmind/mujoco_menagerie

Appendices: Crazyflie Hardware Testing and Configuration

We tested on the Crazyflie 2.1 drones made available by the University of Ottawa. Some of the drones required configuration and/or adjustments, we list the main ones below. Drones are labeled based on the last two digits of their configured radio address, for example, the drone with radio address //0/80/2M/E7E7E7E701 is labeled Drone 01. Drones were operated over concrete with control overrides for safety, as the flow decks could not get accurate position readings over the dedicated netted black mat surface in the test room.

Drone 07 and 03 → Incorrect boot LED indicator failure, cannot connect to Crazyflie

- M2 lights up, M3 blinks rapidly and lights up, both blue
- M1 not blinking red (all is not good on startup)
- Likely running ArduPilot firmware and require flashing to Bitcraze firmware
 - Not tested

Drone 01 → Pins were soldered to the drone, preventing Flow deck connection

- Removed soldered pins, but cannot find flow deck
 - DECK_DISCOVERY: Backend found: onewire
 - DECK_DISCOVERY: Backend found: deckctrl
 - DECK_CORE: 0 deck(s) found
- Reflashed using 2025.2 firmware
 - DECK_CORE: 1 deck(s) found
 - DECK_CORE: Calling INIT on driver bcFlow2 for deck 0
 - ZR2: Z-down sensor [FAIL]
 - PMW: Motion chip id: 0x:0x
 - [...]
 - Error while initializing the PMW3901 sensor
 - DECK_CORE: Deck 0 test [FAIL].
- Re-soldered the pins
 - Drone now recognizes flow deck and operates as expected

Drone 02 → Inconsistent flow deck connection

- Occasionally has connection issues
- Occasionally shows DECK_CORE issue seen in Drone 01 (before re-soldering)

Drones 01 and 02 were the main drones used to test our policies.